

Q1.

- (a) A, C, D, E
- (b) A, B, C, D, E
- (c) C, D
- (d) A, C, E
- (e) A, B, C, D, E

Q2.

- (a) True: A process is created when its PCB is created during the fork() system call. System call execution always happens in the kernel-mode.
- (b) True: Two threads of a process share the same address space. A spin-lock data structure can be created in the shared memory region along with lock and unlock functionalities to ensure correct synchronisation without taking any OS assistance.
- (c) False: Any example where #of page faults with LRU $\geq$ #of page faults with FIFO
- (d) False: Even with synchronous writes, a failure occurring between I/O operations may result in inconsistencies. For example, during a file creation, if the updated directory content is written to disk and a failure occurs before the inode bitmap update, the FS will be in an inconsistent state. (Other examples are fine)
- (e) True: If the infinite loop contains I/O operations (e.g., read, write etc.), then the process goes to waiting mode and even with a non-preemptive scheduler another process will be scheduled. (Arguing the opposite should explicitly mention that there are no I/O operations in the loop)

===== Part II
=====

Q1. The expected answer should address the following concerns

- The page table of child and parent would be modified to mark the mapping as read-only
 - When should it be done?
 - How exactly implemented? Which page table layers are modified?
 - What about TLB flushes?
- The page fault handler should be modified to handle the faults because of write to shared page
 - What is the modified page fault handler logic?

- How a normal RO page differentiated from a shared RO page?
- The munmap(), mprotect(), exit() system calls and swapping
 - How are they implemented?
 - How an unmapped PFN freed?

Q2. Open ended question. Will be evaluated subjectively.

Q3.

(a) Maximum address covered by three L1 entries = $3 * 512\text{GB}$

(b) Min = 0 B, as none of the unmapped address may be mapped [EDITED]

$$\text{Max} = 2\text{GB} + 1030 * 4\text{KB}$$

Explanation:

#of entries freed in L4 = $2^{19} \Rightarrow$ max #of L4 page table pages freed = $2^{10} + 1 = 1025$

#of entries freed in L3 = 1025 $\Rightarrow$ max #of L3 page table pages freed = 3

#of entries freed in L2 = 3 $\Rightarrow$ max #of L2 page table pages freed = 2

#of entries freed in L1 = 1 $\Rightarrow$ max #of L1 page table pages freed = 0 (can not be freed!)

(c) 12 bits

$$512\text{PB} = 2^{59} = 2^{47} \text{ PFNs} \Rightarrow 47 \text{ bits for the PFN address}$$

5 bits for flags

$$\text{Remaining} = 64 - 47 - 5 = 12$$

(d) $(2^9 + 1 + 1 + 1) * 4\text{KB} = 515 * 4\text{KB}$

Explanation: $1\text{GB} = 2^{30} = 2^{18}$ pages which require 2^{18} entries in L4

Min. page table pages in L4 = 2^9

Min. page table pages for 2^9 entries in L3 = 1

Min. page table pages for L2 and L1 is 1 each

(e) $(2^{18} + 2^{18} + 2^9 + 1) * 4\text{KB}$

Explanation:

L4: 2^{18} entries can be scattered across 2^{18} page table pages

L3: 2^{18} entries can be scattered across 2^{18} page table pages

L2: 2^{18} entries can be spread across 2^9 page table pages (as that is the maximum #of pages at this level)

L1: 2^9 entries will consume one page

Q4. Multiple solutions possible. The key is to ensure correct order when concurrent insertion occurs after any particular node. Will be evaluated based on correctness and efficiency.

Q5.

(a) $(8 + 2 * 1024 + 8 * 1024 * 1024) * (4096/256)$ // 256 byte per entry

(b) Min: 1 (at least one block to be written)

Max: $8 + 2 + 2 * 1024 + 8 + 8 * 1024 + 8 * 1024 * 1024 + 1$ // We have to read the whole directory and write one, reading the directory involves reading all the meta-data

(c) Min: 1MB + 4KB // Access is at offset 0, only one single indirect block is read

MAX: 1MB + 16KB. [EDITED]

Data = 1MB + 4KB (read may not be page aligned)

MetaData = 4KB (Double indirect block, one block at DI level) + $2 * 4KB$ (Double indirect block, two SI level blocks)

(d) Minimum: $(5 + 1)$ We find the directory entry in the first block of the directory at each level (one more for file data access)

Maximum: $5 * (8 + 2 + 2 * 1024 + 8 + 8 * 1024 + 8 * 1024 * 1024) + 1$

Explanation: At each level of directory, the entry is found in the last block

=====

If you have any concerns, post it by 3PM tomorrow. Create a followup here.